

ImagoEngine

Technical Design Document

Phase 1: Design, Planning, and Learning

Version 0.1.0

Author: Shane David



SD CREATIONS
CALLED TO CREATE

Contents

1 Project Overview	4
1.1 Mission and Creation Vision	4
1.2 Name and Identity	5
1.3 Goals	5
1.3.1 Data Oriented Design	5
1.3.2 Modularity	5
1.3.3 Future-Proof	5
1.3.4 Full Ownership	5
2 Technology Stack	6
2.2 Library List and Versions	6
2.3 Build System and Toolchain	6
2.3.1 Compiler and Build Tools	6
2.3.2 Package Management (vcpkg)	6
2.3.3 Editor and Extensions	7
3 Game Class and Engine Architecture	8
3.1 Game Class	8
3.2 Input System	9
3.3 SceneContext	10
4 ECS Core	10
4.1 Philosophy and Design Principles	10
4.2 Entities	11
4.3 Component Pooling	13
4.4 Surveys and Bonds	14
4.5 Broadcaster	17
4.6 Systems	18
4.7 System Scheduling	19
4.8 Command Buffer	20

4.9 Nexus	21
4.10 Scene Management.....	23
5 Components and Systems	25
5.2 Core Engine Components	25
5.2 Core Engine Systems	28
6 Asset Management.....	29
6.1 Philosophy	29
6.2 Asset Manager	29
6.3 Asset Wrappers	30
6.4 Asset Components	33

1 Project Overview

ImagoEngine is developed and owned by SD Creations and will be released as a commercial project. It is built on the belief that all people are called to create and serves as the foundational technology through which SD creations will develop and publish its 2D game titles. ImagoEngine is a self-contained, 2D game engine built in C++ using SFML as its rendering and windowing backbone. It is designed around a strict Entity-Component-System (ECS) architecture, providing a clean separation between data and logic that enables high-performance, cache-friendly game development, and simpler design for the often "fast and loose" style of game programming. The engine ships with an integrated visual editor built on Dear ImGui, a custom physics and collision system, and a JSON-based level serialization pipeline. The architecture is deliberately designed to support a scripting layer in a future iteration without requiring structural changes to the core engine.

1.1 Mission and Creation Vision

SD Creation's mission statement:

"At SD Creations we believe that all people are made in the image of a good Creator and are therefore called to create. We pursue that purpose by designing software that entertains, educates, heals, and enables people to pursue their dreams."

Imago Engine's role in that mission:

ImagoEngine is the foundational technology of SD Creations and the professional and quality standard we are building upon. Built from the ground up in C++, ImagoEngine is a fully custom 2D game engine designed around a strict Entity-Component System architecture. This architecture is not an internal optimization hidden from the user but instead is the primary programming paradigm exposed at every layer of the engine including the scripting layer. We believe ECS is not just a performance pattern but fundamentally a better way to design games. By eliminating the traditional game objects and inheritance-based hierarchies in favor of pure data components and stateless systems, ImagoEngine encourages cleaner, more maintainable, and more scalable game architecture from the very first line of code. ImagoEngine will be professional quality, user-friendly, and released

as commercial software. It will also serve as the primary technology SD Creations will use for all their 2D games.

1.2 Name and Identity

Its name comes from the Latin, Imago Dei - the image of God, rooted in our core belief that every person carries within them a reflection of their Creator, and that is fully expressed through the act of creations itself. ImagoEngine does not only serve as the foundation for us to create, but also as an enabler for others to create.

1.3 Goals

1.3.1 Data Oriented Design

ImagoEngine will be designed in such a way that anyone with any experience can use it and quickly learn to understand the importance of data oriented design. ImagoEngine aims to combat the design and performance problems with OOP and inheritance-based Game Objects by replacing it with ECS and DOD.

1.3.2 Modularity

Every engine and editor feature is an isolated system that register against the ECS world and can be swapped or extended independently.

1.3.3 Future-Proof

The ECS API and component model are designed cleanly enough that a scripting runtime can be integrated as an additional system layer down the road without refactoring the core.

1.3.4 Full Ownership

Physics, collision and core systems are written in-house to avoid black-box dependencies and allow deep customization for users.

2 Technology Stack

2.2 Library List and Versions

Library	Version	Purpose
SFML	3.0.2	Rendering and Core
SDL2	2.32.10	Controller reading
imgui	1.91.9	User Interface
imgui-sfml	3.0	Integration with SFML
nlohmann-json	3.12.0	JSON serialization
yaml-cpp	0.9.0	YAML config files
spdlog	1.17.0	Error and debug logging
glm	1.0.3	Complicated math algorithms
Catch2	3.13.0	Unit testing
stb	2024-07-29	Image support
fmt	12.1.0	String formatting

2.3 Build System and Toolchain

2.3.1 Compiler and Build Tools

- Compiler: GCC via MSYS2/MinGW (x64)
- Build system: Ninja
- CMake version: 3.20 minimum
- Shell: Git Bash as default terminal
- Target triplet: x64-mingw-dynamic

2.3.2 Package Management (vcpkg)

- Mode: Manifest mode (vcpkg.json)
- Baseline hash: dd306f32e07d87fdb16837af64f33b6b415c770a
 - The baseline hash locks the entire dependency graph to the state of the vcpkg registry at the most recent commit. It stops vcpkg from resolving to the latest available version each time the project is loaded to prevent it from breaking in-between builds.
- Triplet: x64-mingw-dynamic

vcpkg.json:

```
{
  "name": "imagoengine",
  "version": "0.1.0",
  "builtin-baseline": "dd306f32e07d87fdb16837af64f33b6b415c770a",
  "dependencies": [
    "sfml",
    "sdl2",
    {
      "name": "imgui",
      "features": ["docking-experimental"]
    },
    "imgui-sfml",
    "nlohmann-json",
    "yaml-cpp",
    "spdlog",
    "glm",
    "catch2",
    "stb",
    "fmt"
  ]
}
```

2.3.3 Editor and Extensions

- Editor: VS Code
- Extensions:
 - CMake Tools
 - C/C++ IntelliSense
 - CodeLLDB
 - GitLens
 - Git Graph
 - Todo Tree
 - Better Comments
 - Clang-Format
 - Clang-Tidy
 - Shader support
- Code style configuration:
 - .clang-format settings:
 - Brace style: Attach
 - Indent: 2 Spaces
 - Column limit: 100 characters
 - Pointer align: Left
 - .clang-tidy checks:

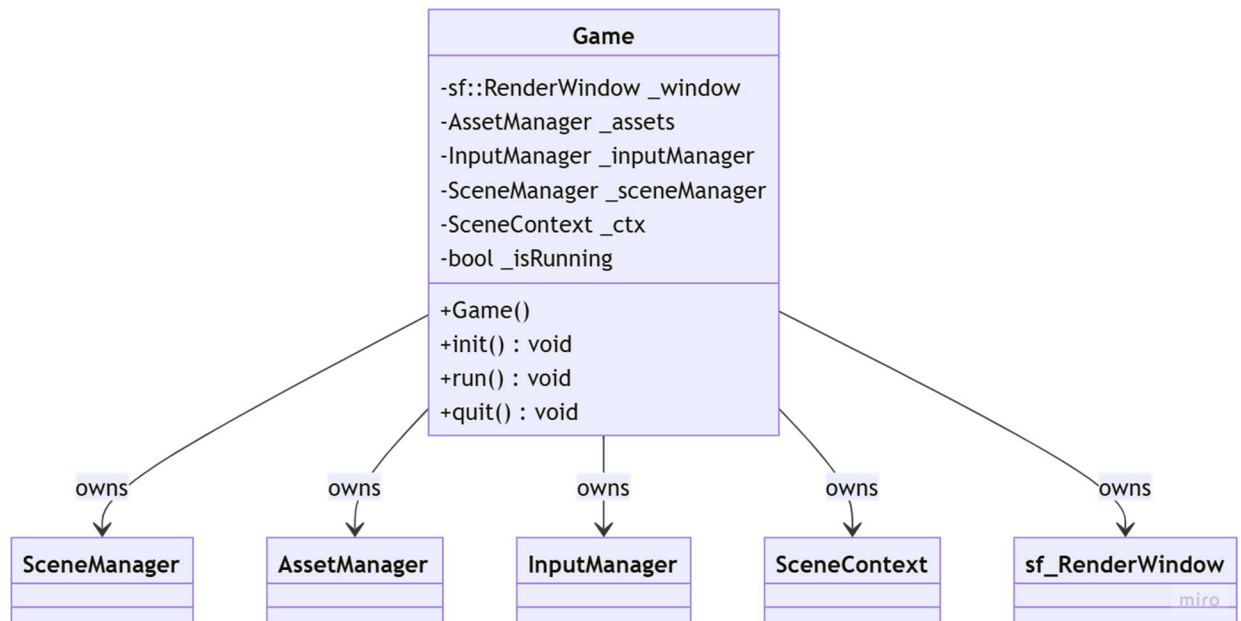
- modernize
- performance
- readability
- bugprone
- cppcoreguidelines
- Naming Conventions:
 - Classes and functions: PascalCase
 - Variables: camelCase
 - Private members: underscore prefix
 - Constants: UPPER_CASE
 - Components: cComponentName prefix

3 Game Class and Engine Architecture

3.1 Game Class

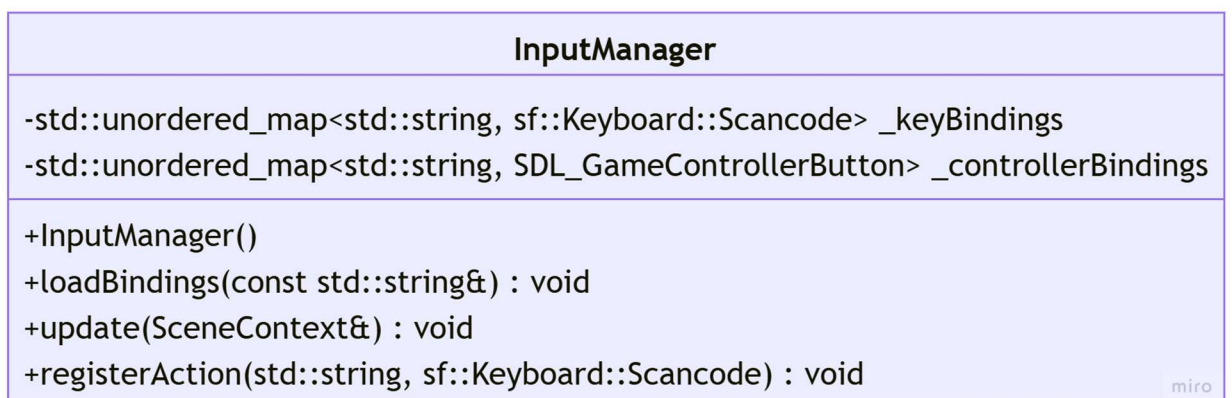
The Game class operates at the lowest level and manages the operations that allow Scenes to run.

- It initializes all persistent engine objects like the window, asset manager, input manager, scene manager, and scene context struct. It then registers all scenes that have been created and loads the first scene so the game is ready to run.
- It keep track of a `isRunning` variable and contains the main while loop that keeps the game running. It also sets this variable to false to terminate the game when the player closes the window or a quit event is emitted.
- Similarly it handles window creation and management. It creates and owns `sf::RenderWindow` and handles any events or data relevant to that object.
- Game also listens for a scene change event so that it can call the `switchScene` method on the `SceneManager`.
- Game also constructs the `SceneContext` from its owned objects and passes it down through the `SceneManager` to the active Scene. this ensures no scene or system every needs to reference the game directly.



3.2 Input System

One of Game's biggest roles is handling user input at the lowest level. It uses SFML and SDL2 event polling to sense for any input from connected keyboards or controllers. If it senses this input it checks it against an action map in the `InputHandler` class that contains maps from input codes to actions. If the input has a registered action the `InputHandler` updates a `std::unordered_map` from strings to Booleans in the `SceneContext` struct that reflects whether or not the input is currently occurring. The user can then use this map in any system to check the input. Input to action binds will occur in the editor and that action will automatically update the registry in the `InputHandler` and add the key to the `input` map in `SceneContext`.



3.3 SceneContext

The `SceneContext` struct will allow the user to access important engine objects (Nexus, asset manager, broadcaster, window, and input action map) within systems. For example, the render system will need to access the `AssetManager` so that it can get textures and set them to each sprite before it is drawn. The Game will create the `SceneContext` originally and fill it with all of the persistent Engine objects that it owns (window, assets, and input map). The objects will be pointers so that it can leave any objects it does not own as `nullptr` and those can be filled in as it passes `SceneContext` down all the way to the Scene which fills in the nexus and broadcaster in order to fully populate `SceneContext`.

<<struct>> SceneContext	
Nexus*	nexus
AssetManager*	assets
Broadcaster*	broadcaster
sf::RenderWindow*	window
std::unordered_map<std::string, bool>*	input

4 ECS Core

4.1 Philosophy and Design Principles

ImagoEngine will implement ECS not only for its internal systems and editor design but also as the main coding paradigm for user development. One of the main missions of ImagoEngine is to encourage the knowledge and use of Data Oriented Design in an age where optimization is often forgotten. ImagoEngine's ECS module focuses on cache efficiency by decoupling data

and logic while maintaining simple and understandable syntax so that beginners can learn and implement ECS.

Along with better optimization ImagoEngine implements ECS to encourage better design and structure in game programming. In contrast to most standard Game Engines, ImagoEngine rejects inheritance-based game object hierarchies entirely. Instead, ImagoEngine ECS limits fragile and monolithic design by eliminating the use of traditional "game objects" and instead only allowing the definition of simple components and systems, making design easier to reason about, extend, and debug at any scale. Similarly it solves the "God Object" and "Spaghetti Code" patterns that are common in quick and unplanned game design by relaying all important data through the Nexus and implementing a Broadcaster to handle all communication between systems.

Syntax should be unique from popular ECS libraries and explicit to the true meaning of an actor's role. Syntax will be clearly different from any other popular game design terms.

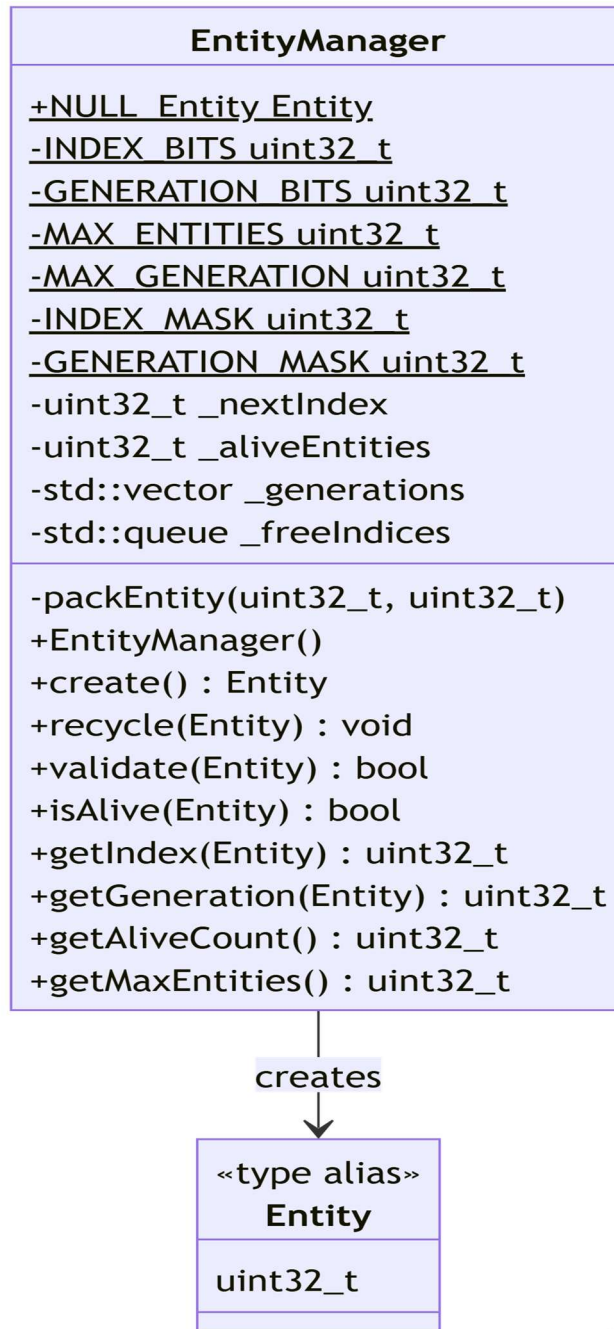
ImagoEngine's ECS will be designed to support a future scripting layer designed specifically for the use of DOD in Game Design.

4.2 Entities

`Entity` is simply an alias for `uint32_t`, in other words an `Entity` in ImagoEngine is nothing more than a number, anything else important to it (names, data, size, sprites, ect.) is contained in the components that are "attached" to it via the Nexus.

```
using Entity = uint32_t;
```

Despite the Entity's simple nature, it is unwise for them to be dealt with explicitly. Instead ImagoEngine ECS will implement an `EntityManager` to create, recycle, and validate entities. Entities will be split into two sections, their index, and their generation. The index will be essentially the identifier of the entity, they are used to access that entity's components. However since id's are removed and reused it is important to change the index when it is reused so that the entity never silently points at the wrong thing. The first 20 bits of the Entity will be their index, and the last 12 bits will be their generation id. The `EntityManager` will internally deal with all of the bitwise math, so the user never has to concern themselves with this.



- `NULL_ENTITY` is a value that represents an invalid entity
- Note on bitwise math:
 - `INDEX_BITS` - how many bits represent the index
 - `GENERATION_BITS` - how many bits represent the generation
 - `MAX_ENTITIES` - how many entities can be represented by the index bits ($1 \ll \text{INDEX_BITS}$)
 - `MAX_GENERATION` - how many generations can be represented by the generation bits ($1 \ll \text{GENERATION_BITS}$)

- `INDEX_MASK` - bit pattern used to isolate the index (`MAX_ENTITIES - 1`)
 - `entity & INDEX_MASK` - zeros out the generation
- `GENERATION_MASK` - bit pattern used to isolate the generation (`~INDEX_MASK`)
 - `(entity & GENERATION_MASK) >> INDEX_BITS` - zeros out index and shifts to a readable number

4.3 Component Pooling

The `sparseSet` is the data structure that will accomplish “Component Pooling” for ImagoEngine ECS. ECS aims to accomplish cache efficiency by storing the actual objects contiguously in memory, but in game development, preserving that is constantly threatened because entities are constantly being created, destroyed, and reused, Sparse Sets solve this problem with three arrays working together.

The sparse array (`_sparse`) is indexed by Entity Ids. It is the size of the maximum number of entities and stores either the index into the component list for that entity, or `NULL_INDEX` if that entity does not exist or does not have that component. For example, if entities 1, 2, and 5 all have Transform components then `_sparse` will contain a 0, 1, and 2 at those indices and `NULL_INDEX` everywhere else.

```
[NULL_INDEX 0 1 NULL_INDEX NULL_INDEX 2]
```

The dense array (`_dense`) is tightly packed, meaning it contains no holes, just a list of components. It is indexed by the indices in `_sparse`. For example, if entities 1, 2, and 5 all have Transform components then `_dense` will contain the Transform components for those entities. When a component is removed the Sparse Set swaps it with the last element, pops it, and updates the sparse and entity arrays in order to keep it tightly packed.

```
[T1 T2 T5]
```

The entity array (`_entities`) is also tightly packed and it mirrors the dense array; it is just a list of all the entities that have the respective component for that Sparse Set. For example, if entities 1, 2, and 5 all have Transform components then `_entities` will simply contain 1, 2, and 5.

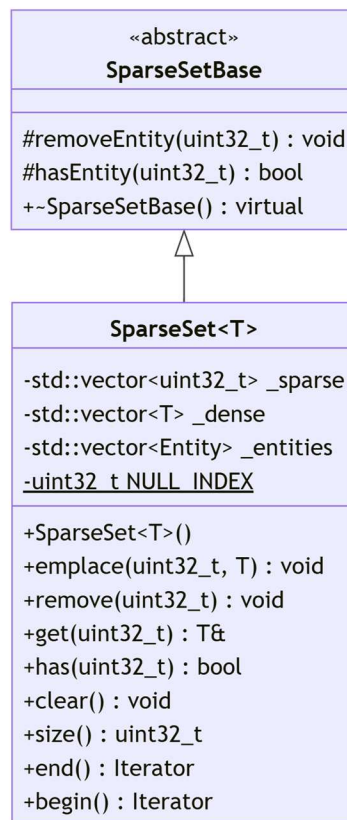
```
[1 2 5]
```

This system allows for `.get()` to be called on an Entity. The Entity (which is just an integer) will index the sparse array which will return the index in the dense and entity array for that entity. Those arrays can then be indexed to get the entities' respective component.

```
uint32_t denseIndex = _sparse[entityIndex];
T& component = _dense[denseIndex]
Entity e = _entities[denseIndex]
```

This system allows systems to still loop through components without any cache misses because they are stored contiguously in an array with no gaps.

The `SparseSetBase` abstract class exists to give all `SparseSet`'s a common type so that they can be stored in the Nexus.

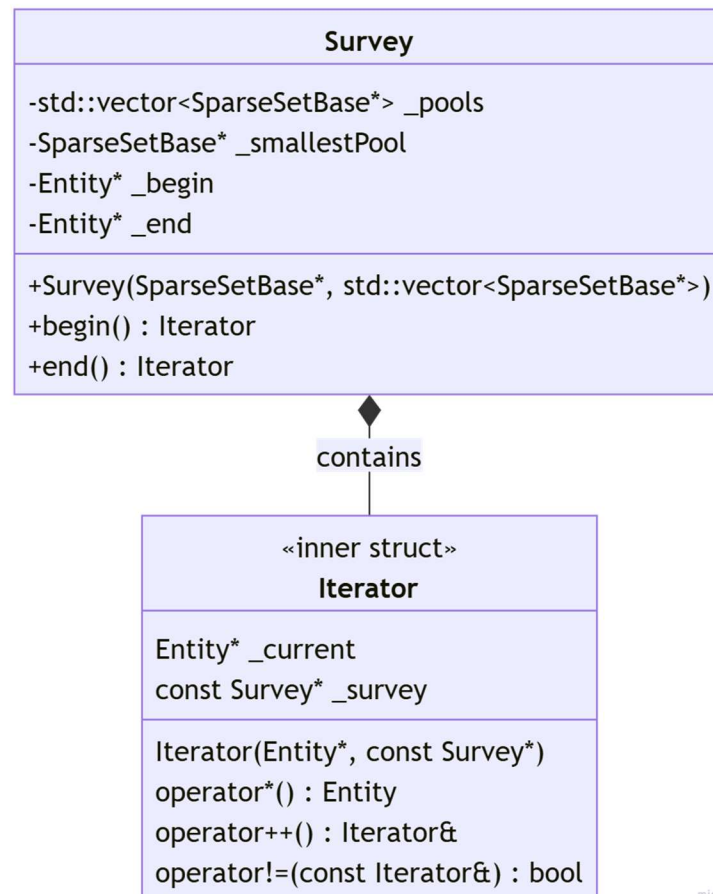


4.4 Surveys and Bonds

Surveys are the most common method for iterating over entities. They are temporary objects that are created fresh every frame. When a survey is created it "surveys" all the requested component pools and is created as an iterator that references the smallest one. Then, while iterating the Survey

steps through the smallest pool and checks if each entity is also in the other requested pools so that only entities that exist in every requested pool are returned within the systems. This allows for systems to iterate over entities with specific components like so:

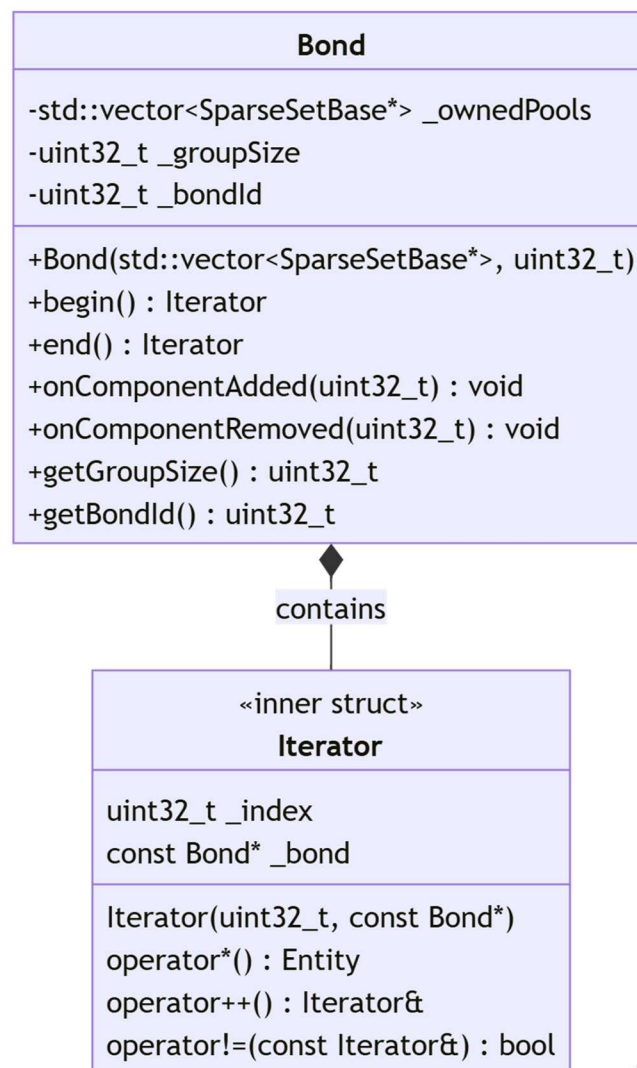
```
void CollisionSystem(SceneContext& ctx, float dt) {
    auto survey = ctx.nexus.survey<cCollider, cRigidbody>();
    for (Entity e : survey) {
        // do collision logic on e
    }
}
```



Bonds are used in the same manner as Surveys (to iterate through entities with certain components) however, as the name Bond suggests, they are more permanent. Bonds are created once and live forever inside of the Nexus so that they can maintain a presorted pool layout. They now actually own the Component pool for whichever components the Bond exists for and sort the **SparseSet** so that the entities that share all required components are at the start of the dense array. In order to maintain this the Nexus must notify the Bonds whenever a component is bound or unbound so that the

Bond can update its sorted region. This notification is why using a Bond for every system is not practical, the iteration tradeoff does not overcome the need for the notifying and rearranging for every component change. This allows for iteration to be faster because Bonds do not have to do membership checks but there is a higher set up cost. This is better for systems that are always running on a large and frequently changing set of entities. (Collision, Rendering, Physics).

NOTE: if a Bond exists for a component type, there CAN NOT exist another Bond for that component type. For example: there cannot be `Bond<cTransform, cCollider>` and `Bond<cTransform, cSprite>`



4.5 Broadcaster

The broadcaster serves the same purpose as an `EventManager` would in typical object-oriented game design, however it is more tightly integrated with systems.

When an important event happens, whichever system detects the event can use the `Broadcaster` to "broadcast" that event. Subscribed Systems "subscribe" to said event and when they receive the broadcast they are called. Events are plain data structs defined by the user that contain any data needed to execute the logic in whatever system subscribes to that event. Subscriptions happen during the initialization phase of each scene, so there is a different `Broadcaster` instance for each scene.

This system exists so that systems never have to reference each other and each system can be decoupled. Instead of the collision system needing to know everything about health and damage logic it can just broadcast a collision event that the damage system subscribes to and deals with accordingly.

The broadcaster works by storing callbacks per event type when an event is registered and calls them when an event is emitted. It is owned by Scene and constructed with a `SceneContext` reference so that it can pass it to Subscribed Systems.

Broadcaster
-SceneContext& _ctx -std::unordered_map<EventTypeId, functionPointers>_subscribers -uint32_t _nextEventTypeId
+Broadcaster(SceneContext&) +emit<T>(T& event) : void +receive<T>(SystemFn) : void +ignore<T>(SystemFn) : void +clear() : void

miro

- `functionPointers` refers to the list of type erased callbacks:
`std::vector<std::function<void(void*)>>`, `void*` is cast back to the correct event type `T` at call time inside `emit`
- `SystemFn` refers to the type alias for a subscribed system function signature: `std::function<void(SceneContext&, T)>`

`EventTypeId` will be simply an alias for `uint32_t` and will be used to index the map of subscribers. In order to go from the event passed in to the `EventTypeId` `ImagoEngine` will implement the Meyers Singleton pattern. The pattern works by having one function that has a static variable to keep track of the ids and increments the next id whenever one is created. It then has a templated function that initializes the id and returns it. Because the function is templated the id variable has a unique instance for each type (Event) passed in when the function is called. Since the id is static C++ will only initialize it once so when the function is called for the first time it initializes the id but when the same function is called in order to get the id it solely returns it. This pattern will be used again for the Nexus to access Component Pools.

```
inline uint32_t nextId() {
    static uint32_t id=0;
    return id++;
}

template<typename T>
uint32_t getId() {
    static uint32_t id = nextId();
    return id;
}
```

4.6 Systems

Systems are the only place that logic is allowed in `ImagoEngine` ECS. Systems will simply be free functions defined by the user in their own files. Systems will receive any needed information through a `SceneContext` reference which will provide access to the `Nexus`, `AssetManager`, `Broadcaster`, `RenderWindow`. Users who require more global game state should use a singleton entity pattern in order to store global data in a component on a dedicated entity accessible through the Nexus. The user can then use any function with the proper signature to perform any game logic they require.

Defining systems like this allow for systems to be reusable across multiple scenes since any scene can register the same system without duplicating logic.

Most often systems will take advantage of `Surveys` and `Bonds` in order to perform logic on a set of entities that contain a specific component or set of components.

Some systems like rendering and collision need to be updated every frame, these systems will be referred to as Scheduled Systems, they deal with the `SystemScheduler` and run every frame in whichever scene they ought to run.

Other systems like damage and trigger collisions just need to be called whenever a specific **event** happens in the logic of the game. These will be referred to as Subscribed Systems, they deal with the `Broadcaster` and are only called when the event that they have subscribed to is emitted by another system.

System signatures will be enforced through editor code generation. When a user creates a new system through the ImagoEngine editor interface it will automatically create a `.cpp` file that has a correctly signed function stub.

The system signatures will look as follows:

```
// Scheduled System
void exampleSystem(SceneContext& ctx, float dt)
// Subscribed System
void exampleSystem(SceneContext& ctx, EventType& event)
```

4.7 System Scheduling

The `SystemScheduler` is an important object within the context of the scene. Scheduled systems (systems that need to happen every single frame) need to execute in a specific order in order to maintain correct game behavior. Built in systems like physics, collision, animation, rendering, audio ect. will run in a fixed predefined order and cannot be reordered or modified under normal usage. However, if the user wants to operate on the lower level they can choose to disable those systems and create their own, hence exposing the `SystemScheduler` at a lower level so they can schedule core systems.

The `SystemScheduler` will also allow for the user to choose the order in which their own Scheduled Systems go in but this will not be exposed in code: the user will simply click and drag their desired order in the editor. These will be run after core systems.

SystemScheduler
-std::vector<SystemFn> _coreSystems -std::vector<SystemFn> _userSystems -std::vector<SystemFn> _fixedSystems -SceneContext& _ctx -float _accumulator -float _fixedTimestep
+SystemScheduler(SceneContext&) +tick(float dt) : void +registerUpdate(SystemFn) : void +registerFixed(SystemFn) : void +unregister(SystemFn) : void +setFixedTimestep(float) : void +setUserOrder(std::vector<SystemFn>) : void

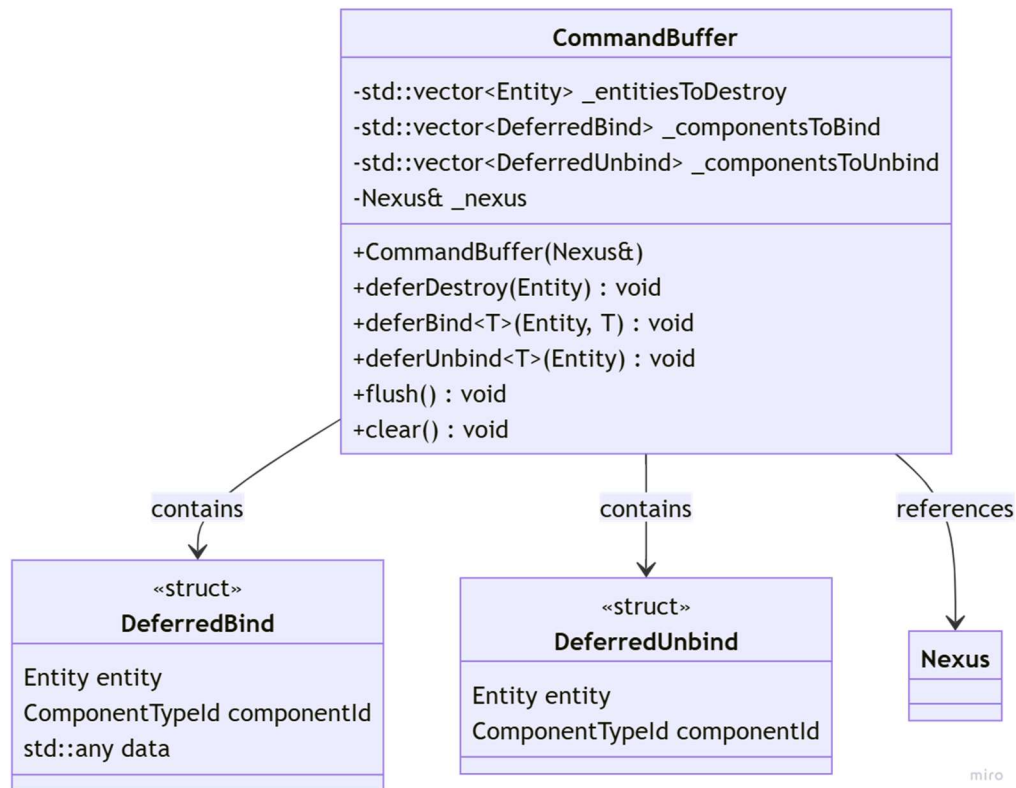
The `tick` method of the `SystemScheduler` is called once per frame by the Scene. It then iterates through the ordered core systems and user systems calling each systems function in order. In fixed timestep systems (that should not rely on framerate) it accumulates real delta time with the accumulator and only calls them when enough time has built up for a complete fixed step.

4.8 Command Buffer

The `CommandBuffer` solves a crucial issue in the iterative nature of ECS systems. Since everything is stored in arrays, those arrays are being iterated over for the entire duration of the frame. When the arrays need to be updated, their updates ought to wait until the end of the frame because modifying a container while it is being iterated over can cause unexpected behavior.

Whenever an entity is destroyed or structurally changed the action is deferred to the `CommandBuffer` automatically and then at the end of each frame the `CommandBuffer` makes the actual changes to the Sparse sets.

Things like entity destruction, component binding, and component unbinding are deferred to the `CommandBuffer` while things like reading and writing component data, creating entities, and emitting events do not because they do not modify any arrays.



The `flush()` method in the `CommandBuffer` iterates through the vectors for binds, unbinds, and destructions and actually performs the modifications via the Nexus reference.

4.9 Nexus

The Nexus is essentially the brain of the ECS world, it contains all of the Component Pools, and the Entity Manager and is used for creating Entities, binding and unbinding components, and creating Surveys and Bonds. The Nexus owns all Bonds and notifies them whenever a component is bound or unbound so they can maintain their sorted regions.

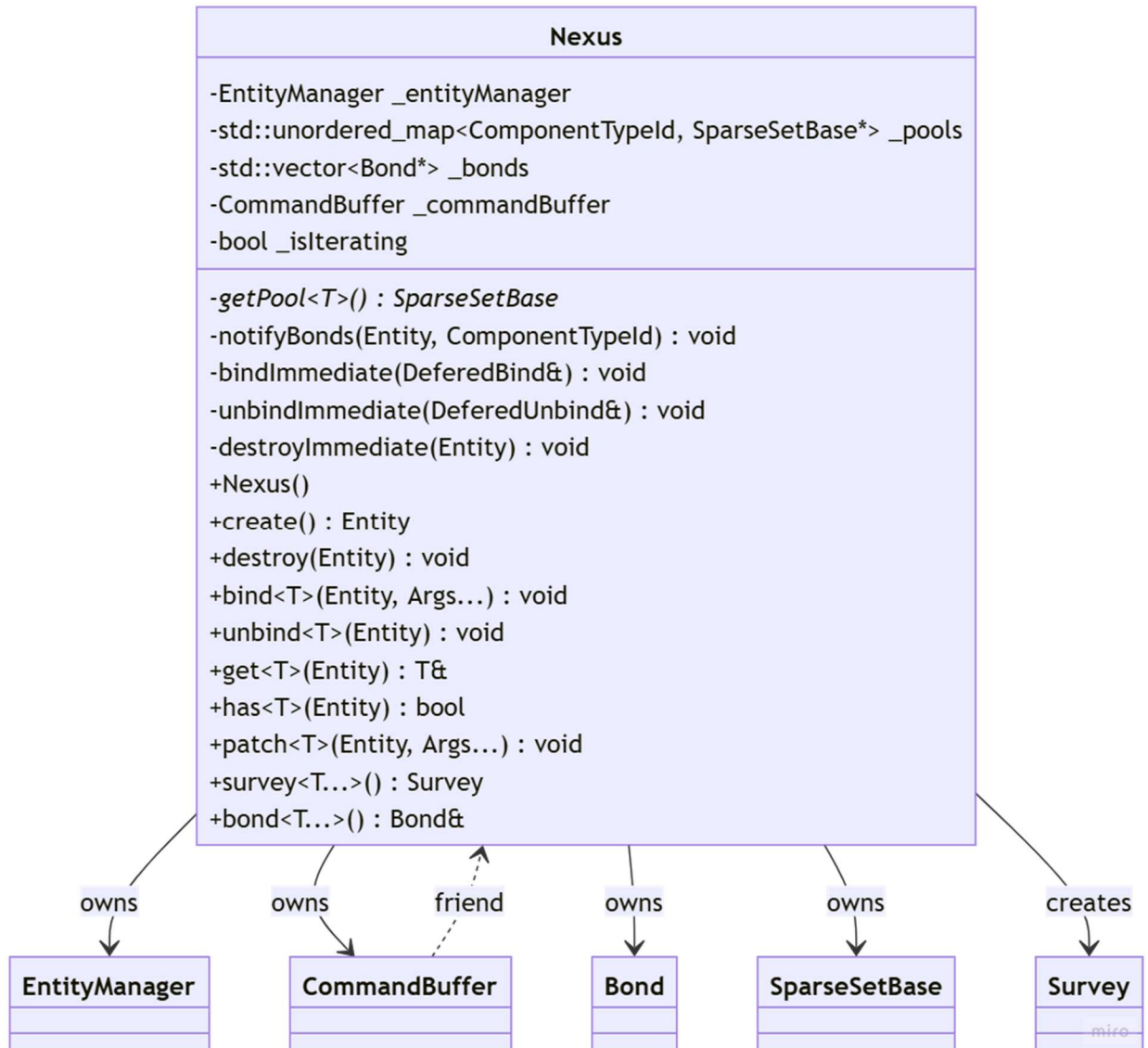
The point of the Nexus is to contain all of the Entity & Component related methods and to internally deal with all validation and low level processes so the user only has to worry about a single call.

The Nexus owns the `CommandBuffer` internally and it will defer all structural changes during iteration to it instead of the user having to call the `CommandBuffer` directly. The `CommandBuffer` is also a Friend Class of the Nexus so that it can access the private methods for binding, unbinding, and removing immediately during a flush.

The Nexus also validates every Entity ID before performing any operation on it.

Every system has access to the Nexus via `SceneContext`.

The Nexus uses the same Meyers Singleton pattern established for `EventTypeId` in the `Broadcaster` but this time to map component types to their respective `SparseSets` via a `ComponentTypeId`, allowing templated methods like `bind<T>` to find the correct pool at runtime.



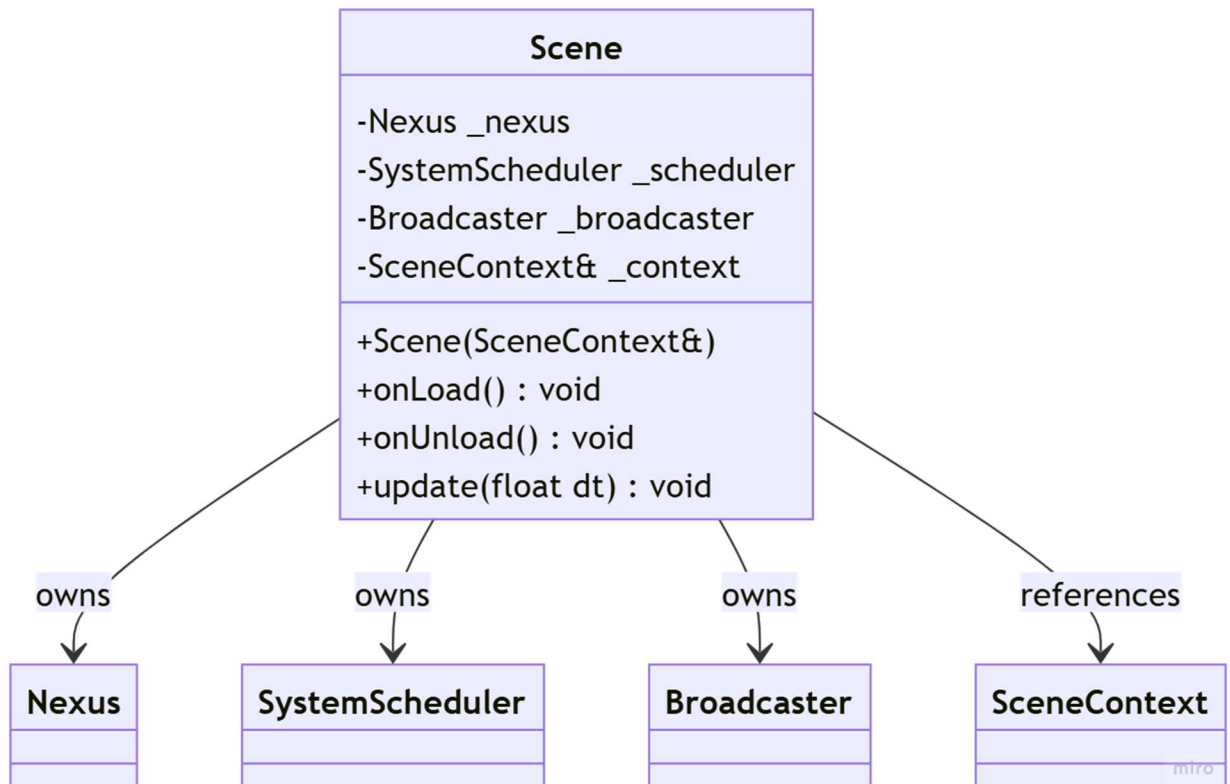
4.10 Scene Management

The `Scene` Class will essentially be the world the Nexus lives in and that systems occur in. It owns the Nexus, `SystemScheduler` and, `Broadcaster`. It will contain the `onLoad` method where the user can create their entities and components, subscribe systems to events and do any other necessary loading.

The `update` method will be unavailable directly to the user but they will be able to add and change the order of the systems called in it via the editor.

The `onUnload` method will also be available to user where they can perform any necessary cleanup. However most common cleanup (destroying Entities, cleaning up Component Pools ect.) is handled automatically by the Nexus.

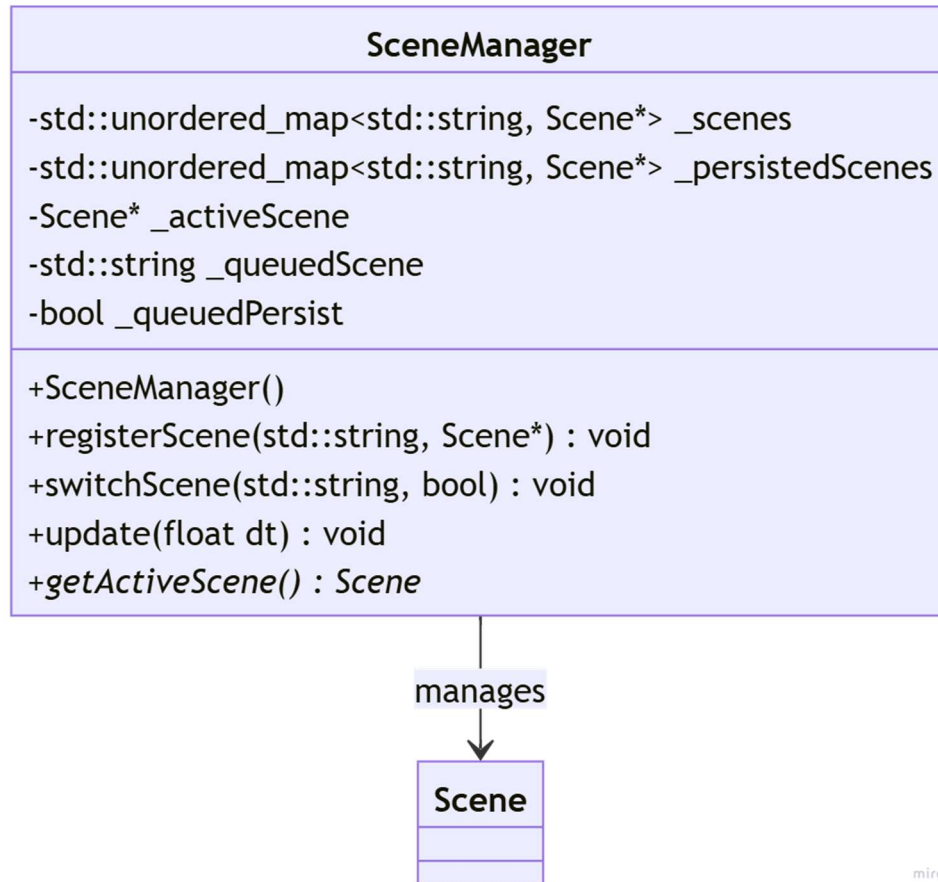
The Scene will receive the `SceneContext` reference from the `Game` class via the `SceneManager`, it will pass this to systems to give them access to the window, `AssetManager`, and other persistent engine objects.



The `SceneManager` will hold all of the Scenes and deal with switching between Scenes. When switching scenes the `SceneManager` can either destroy the current scene and free its resources or persist it in memory so it can be returned to without reloading, the user will be able to decide this behavior.

Users will be able to switch between Scenes by creating `SceneChange` events and passing in the Scene Name and whether or not it should persist. the `Game` will be listening for `SceneChange` events and will call the `SceneManager` in order to deal with it.

The user will be able to register Scenes to the `SceneManager` via the editor.



5 Components and Systems

5.2 Core Engine Components

- cTransform

<<struct>>
cTransform

glm::vec2 position
glm::vec2 scale
float rotation

miro

- cRigidBody

<<struct>>
cRigidBody

glm::vec2 velocity
float mass
float restitution
float gravityScale
float isStatic

miro

- cCollider

<<struct>>
cCollider

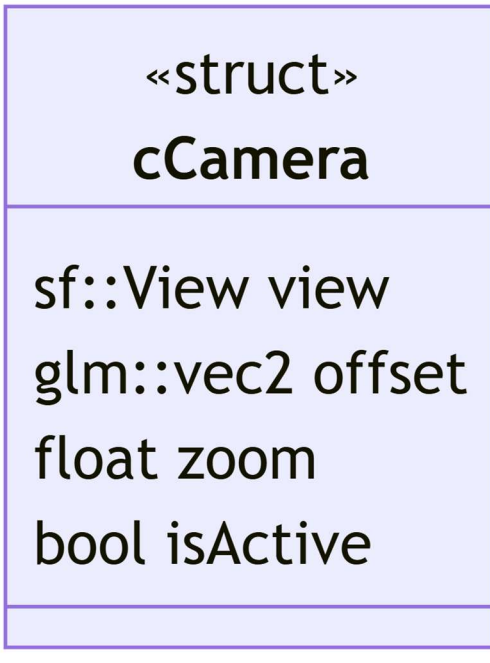
glm::vec2 offset
glm::vec2 size
bool isTrigger

- cTag

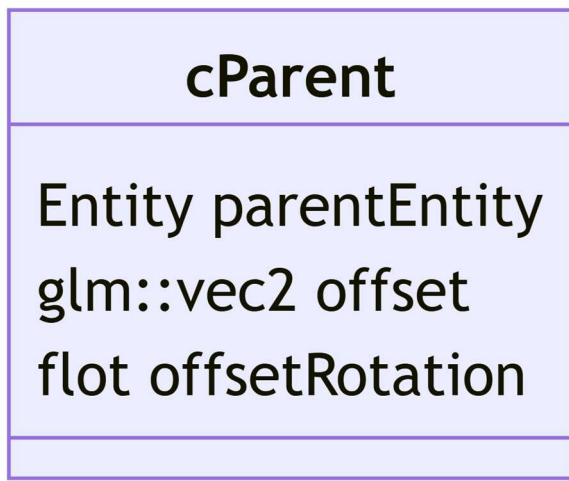
<<struct>>
cTag

std::string tag

- cCamera



- cParent



5.2 Core Engine Systems

NOTE: Flowcharts and more details on systems will be brainstormed and provided once they are integrated into the Core Engine development phase of the project.

System	Type	Reads	Writes	Bond/Survey	Emits
Physics System	Fixed	cTransform, cRigidbody	cTransform, cRigidBody	Bond	n/a

Collision System	Fixed	cTransform, cCollider	n/a	Survey	Collision Event
Render System	Update	cTransform, cSprite, cText, cShape	n/a	Bond	n/a
Animation System	Update	cAnimation, cSprite	cAnimation, cSprite	Survey	n/a
Audio System	Update	cAudioSource	cAudioSource	Survey	n/a
Transform Sync System	Update	cTransform, cParent	cTransform	Survey	n/a
Camera System	Update	cTransform, cParent	cCamera	Survey	n/a

6 Asset Management

6.1 Philosophy

Assets should be loaded once, stored centrally, and referenced everywhere they are needed. Anything that is loaded from disk and shared across multiple entities should be treated as an asset. Anything whose properties are defined by the user and can change per entity should be treated as a component.

6.2 Asset Manager

The Asset Manager owns all the loaded assets and hands out references on request. When something asks for an asset, it either returns a reference, or loads the asset, stores it, then returns a reference.

The difference between `request` and `get` here is important. `request` will be called using the asset path during the initialization of a scene, it ensures that the asset is loaded and returns the id from the paths to id map. `get` will be called using the id stored in the respective asset component and returns the actual asset so that the SFML object of the component can be set to the proper asset.

AssetManager
-uint32_t _nextId -std::unordered_map _textureMap -std::unordered_map _fontMap -std::unordered_map _bufferMap -std::unordered_map _shaderMap -std::unordered_map _texturePaths -std::unordered_map _fontPaths -std::unordered_map _bufferPaths -std::unordered_map _shaderPaths
-loadTexture(const std::string&) : uint32_t -loadFont(const std::string&) : uint32_t -loadBuffer(const std::string&) : uint32_t -loadShader(const std::string&, const std::string&) : uint32_t +AssetManager() +loadStartAssets(const std::string& manifestPath) +requestTexture(const std::string&) : uint32_t +requestFont(const std::string&) : uint32_t +requestBuffer(const std::string&) : uint32_t +requestShader(const std::string& vert, const std::string& frag) : uint32_t +getTexture(uint32_t) : const Texture& +getFont(uint32_t) : const Font& +getBuffer(uint32_t) : const SoundBuffer& +getShader(uint32_t) : const Shader&

miro

6.3 Asset Wrappers

ImagoEngine will define its own classes as wrappers for each SFML asset. The assets defined below are all of the heavy-duty assets that should be loaded to the disk only once and stored and managed by the `AssetManager`.

For the most part they are also shared by several entities. For example, a platform texture is referenced by every platform with that texture in the game, however each one of those platforms has different sprites that contain data specific to that entity.

There are several things missing that "feel" like assets but are not as they are not loaded from the disk and shared across multiple entities.

For example, there is no sprite wrapper because sprites simply reference textures and hold data about what part and in which way to display the texture they reference. In other words, sprites are purely data and therefore perfect candidates for components.

Texture
-sf::Texture _texture -uint32_t _width -uint32_t _height -std::string _path -std::vector _frames
+Texture(const std::string&) +Texture(const std::string&, int, int) +Texture(const std::string&, std::vector) #getWidth() : uint32_t #getHeight() : uint32_t #getPath() : const std::string& #getSFMLTexture() : const sf::Texture& +getFrame(int) : sf::IntRect& +getFrameCount() : int

micro

Font
-sf::Font _font -std::string _path
+Font(const std::string&) +getFont() : const sf::Font& +getPath() : const std::string&

micro

SoundBuffer
-sf::SoundBuffer _buffer -std::string _path
+SoundBuffer(const std::string&) +getBuffer() : const sf::SoundBuffer& +getPath() : const std::string&

Shader
-sf::Shader _shader -std::string _vertPath -std::string _fragPath
+Shader(const std::string&, const std::string&) +getShader() : const sf::Shader& +getVertPath() : const std::string& +getFragPath() : const std::string&

6.4 Asset Components

Asset Components are “attached” to individual entities. They only store data, and their data is specific to the entity, it is never shared.

Note that in the `init` system for any scene, or whenever one of the asset components is added, their corresponding SFML object will have to be initialized at the asset they store and id to.

«struct» SpriteComponent
uint32_t textureId uint32_t frameIndex sf::Sprite sprite sf::Color color bool isVisible int renderLayer
SpriteComponent(uint32_t, uint32_t, sf::Color, bool, int)

«struct» AnimationComponent
uint32_t textureId int startFrame int endFrame int currentFrame int previousFrame float frameDuration float timer bool isPlaying bool doesLoop bool isFinished
AnimationComponent(uint32_t, int, int, float, bool)

«struct» TextComponent
uint32_t fontId sf::Text text std::string string uint32_t fontSize sf::Color color bool isVisible int renderLayer
TextComponent(uint32_t, const std::string&, uint32_t, sf::Color&, bool, int)

